

Содержание

1. Зачем нужен этот файл	3
2. Общая карта проекта	3
3. Главная идея потока выполнения	3
4. Разбор <code>config.py</code>	4
4.1. Почему здесь используется <code>os.environ.get(...)</code>	4
4.2. Группы переменных	4
4.2.1. 1. Базовая воспроизводимость	4
4.2.2. 2. Настройки датасета	4
4.2.3. 3. Размеры выборок	5
4.2.4. 4. Обучение	5
4.2.5. 5. <code>DataLoader</code> и <code>preprocessing</code>	5
4.2.6. 6. Переобучение	6
4.3. Что важно понять по <code>config.py</code>	6
5. Разбор <code>data.py</code>	6
5.1. Импорты и зависимость от <code>config.py</code>	6
5.2. Словарь <code>DATASET_INFO</code>	6
5.3. Словарь <code>DATASET_NORMALIZATION</code>	7
5.4. Функция <code>load_image_dataset(name, data_root)</code>	7
5.5. Функция <code>take_subset(dataset, subset_size, seed=SEED)</code>	7
5.6. Функция <code>split_train_val(dataset, val_ratio, seed=SEED)</code>	7
5.7. Функция <code>make_loaders(...)</code>	8
5.8. Функция <code>plot_random_images(...)</code>	8
6. Разбор <code>cnn.py</code>	8
6.1. Импорты и их смысл	9
6.2. <code>get_device()</code>	9
6.3. <code>set_seed(seed=SEED)</code>	9
6.4. Класс <code>ConvBlockA</code>	9
6.5. Класс <code>ConvBlockB</code>	10
6.6. Класс <code>ImageCNN</code>	10
6.7. <code>is_valid_config(...)</code>	11
6.8. <code>make_optimizer(name, params, lr)</code>	11
6.9. <code>confusion_matrix(y_true, y_pred, n_classes)</code>	11
6.10. <code>train_one_epoch(...)</code>	12
6.11. <code>evaluate(...)</code>	12
6.12. <code>train_model(...)</code>	13
7. Разбор <code>block1_main.py</code>	13
7.1. Импортируемые сущности	13
7.2. Сетка гиперпараметров	14
7.3. <code>build_grid(image_size=28)</code>	14

7.4. <code>short_name(cfg)</code> и <code>cfg_pretty(cfg)</code>	14
7.5. <code>run_one(...)</code>	15
7.6. <code>train_final_on_full(...)</code>	15
7.7. <code>print_results_table(results)</code>	15
7.8. Функции построения графиков	16
7.8.1. <code>plot_curves(...)</code>	16
7.8.2. <code>plot_confusion(...)</code>	16
7.8.3. <code>plot_param_compare(...)</code>	16
7.8.4. <code>plot_grid_summary(...)</code>	16
7.9. Главная функция <code>main()</code>	16
8. Разбор <code>block2_main.py</code>	17
8.1. <code>BASE_CFG</code>	17
8.2. <code>make_model(info, device, dropout=0.0)</code>	17
8.3. <code>OPT_VARIANTS</code>	18
8.4. <code>study_optimizers(...)</code>	18
8.5. <code>plot_optimizer_curves(results, target_acc=TARGET_ACC)</code>	18
8.6. <code>run_overfit_demo(info, device)</code>	18
8.7. <code>plot_overfit_compare(res_no, res_dr)</code>	19
8.8. <code>main()</code> второго блока	19
9. Какие структуры данных гуляют между функциями	19
9.1. 1. <code>info</code>	19
9.2. 2. <code>cfg</code>	20
9.3. 3. <code>history</code>	20
9.4. 4. <code>res</code> или <code>result</code>	20
9.5. 5. <code>final_res</code>	21
10. Один полный проход выполнения блока 1	21
11. Где в коде находятся ключевые решения	22
11.1. Выбор датасета	22
11.2. Выбор типа блока	22
11.3. Выбор числа каналов	22
11.4. Выбор оптимизатора	22
11.5. Решение об <code>early stopping</code>	22
11.6. Выбор лучшей архитектуры	22
11.7. Демонстрация переобучения	22
12. Типичные вопросы, которые стоит задать себе при чтении кода	22
13. Что в этом коде особенно хорошо сделано	23
14. Итог	23
15. Как лучше читать исходники после этого документа	23

1. Зачем нужен этот файл

`main_full.typ` уже даёт теоретическое объяснение лабораторной. Этот документ нужен для другой цели: **разобрать сам код**, чтобы было понятно не только что делает CNN в теории, но и **как именно эта лабораторная реализована в Python**.

Здесь разбор будет идти не от общей теории к идее, а от структуры проекта к реальному выполнению программы:

- какие модули есть в `lab4`;
- что каждый модуль экспортирует;
- как данные переходят из одного файла в другой;
- как строится модель;
- как организован цикл обучения;
- как именно работают `block1_main.py` и `block2_main.py`.

Если читать этот файл параллельно с исходниками, код станет заметно прозрачнее.

2. Общая карта проекта

В `lab4` код логически разбит на четыре уровня.

Уровень	Что находится здесь
Конфигурация	<code>config.py</code> хранит все параметры запуска: размеры выборок, число эпох, <code>patience</code> , <code>dataset name</code> и так далее.
Данные	<code>data.py</code> отвечает за загрузку, нормализацию, подвыборки, <code>split train/val</code> и <code>DataLoader</code> .
Модель и обучение	<code>cnn.py</code> содержит архитектуру CNN, оптимизаторы, метрики, цикл эпохи, валидацию и раннюю остановку.
Сценарии экспериментов	<code>block1_main.py</code> и <code>block2_main.py</code> собирают всё вместе и запускают конкретные исследования.

Это очень правильная структура. Она означает, что автор кода не смешивает:

- настройки;
- загрузку данных;
- описание модели;
- исследовательский сценарий.

Такой код намного легче читать, менять и отлаживать.

3. Главная идея потока выполнения

Если посмотреть на лабораторную сверху, то программа проходит такой путь:

1. Из `config.py` загружаются параметры.
2. `data.py` загружает датасет и подготавливает выборки.

3. `cnn.py` создаёт модель и предоставляет функции обучения/оценки.
4. `block1_main.py` или `block2_main.py` решают, **какой именно эксперимент** нужно выполнить.
5. Результаты печатаются в консоль и визуализируются графиками.

Очень важно держать в голове именно это разделение. Тогда любой вызов функции сразу читается в контексте: это работа с конфигом, данными, моделью или сценарием эксперимента.

4. Разбор `config.py`

`config.py` — это точка, откуда программа получает почти все внешние настройки. По сути это центральное место управления экспериментом.

4.1. Почему здесь используется `os.environ.get(...)`

Почти каждая переменная объявлена по схеме:

```
NAME = os.environ.get('SOME_ENV', 'default_value')
```

Смысл такой:

- если переменная окружения не задана, берётся стандартное значение;
- если задана, код переиспользуется без редактирования файла.

Это полезно для быстрых запусков. Например, можно менять размер подвыборки или число эпох прямо из терминала, а исходный код сценариев останется тем же.

4.2. Группы переменных

Файл разделён на несколько блоков.

4.2.1. 1. Базовая воспроизводимость

```
SEED = 52
```

Этот `seed` потом используется в `numpy`, `torch` и обычном `random`. Его задача — сделать результаты более воспроизводимыми.

4.2.2. 2. Настройки датасета

```
DATASET_NAME  
DATA_ROOT
```

`DATASET_NAME` решает, что именно будет загружено: `MNIST` или `FashionMNIST`.

`DATA_ROOT` указывает, где хранить данные на диске.

4.2.3. 3. Размеры выборок

```
TRAIN_SUBSET
TEST_SUBSET
VAL_RATIO
```

Это очень важные параметры именно для **экспериментального дизайна**.

- `TRAIN_SUBSET` ограничивает размер обучающей выборки для быстрых сравнений архитектур.
- `TEST_SUBSET` ограничивает размер тестовой выборки в промежуточных прогонах.
- `VAL_RATIO` задаёт долю `validation` при разбиении `train`.

4.2.4. 4. Обучение

```
BATCH_SIZE
N_EPOCHS
PATIENCE
MIN_DELTA
TARGET_ACC
FINAL_EPOCHS
```

Этот набор управляет уже динамикой обучения.

- `BATCH_SIZE` — сколько объектов обрабатывается за один шаг.
- `N_EPOCHS` — максимальное число эпох.
- `PATIENCE` и `MIN_DELTA` — параметры ранней остановки.
- `TARGET_ACC` — контрольная целевая ассигасу.
- `FINAL_EPOCHS` — число эпох финального дообучения лучшей модели.

4.2.5. 5. DataLoader и preprocessing

```
NORMALIZE_IMAGES
NUM_WORKERS
PIN_MEMORY
```

Это настройки загрузки данных и поведения `DataLoader`.

Особенно полезно понять строку с `PIN_MEMORY`:

```
PIN_MEMORY = None if _pin_memory_raw == 'auto' else ...
```

То есть код поддерживает режим `auto`, где решение принимать ли `pin_memory`, откладывается до момента создания `DataLoader` и зависит от того, есть ли `CUDA`.

4.2.6. 6. Переобучение

```
OVERFIT_TRAIN
OVERFIT_EPOCHS
```

Эти параметры нужны не для обычного подбора архитектуры, а для специального эксперимента из `block2_main.py`, где на маленькой подвыборке демонстрируется overfitting.

4.3. Что важно понять по `config.py`

Этот файл ничего не обучает и ничего не загружает. Он только **объявляет значения**, которыми потом пользуются другие модули. Это значит, что `config.py` — пассивный источник параметров, а логика находится уже в других файлах.

5. Разбор `data.py`

`data.py` отвечает за всё, что связано с входными данными. Это один из самых важных файлов, потому что любая модель работает настолько корректно, насколько корректно подготовлены данные.

5.1. Импорты и зависимость от `config.py`

В начале файла импортируются:

- `numpy` — для работы с индексами и генератором случайных чисел;
- `matplotlib.pyplot` — для показа примеров изображений;
- `torch` и `torch.utils.data` — для `DataLoader` и `Subset`;
- `torchvision.datasets`, `torchvision.transforms` — для самих датасетов и преобразований;
- из `config.py` берутся `SEED`, `NORMALIZE_IMAGES`, `NUM_WORKERS`, `PIN_MEMORY`.

Это уже важный момент: `data.py` ничего не знает о CNN-архитектуре, зато знает о режиме загрузки данных.

5.2. Словарь `DATASET_INFO`

Это реестр поддерживаемых датасетов. Для каждого имени задаются:

- какой класс загрузчика использовать;
- список имён классов;
- число входных каналов;
- размер изображения.

Например, для `MNIST`:

- `loader` — `datasets.MNIST`;
- `classes` — строки от 0 до 9;

- `in_channels` — 1, потому что изображение чёрно-белое;
- `image_size` — 28.

Такой подход хорош тем, что код ниже не захардкожен на один датасет. Он спрашивает свойства у `info`, а не предполагает их вручную.

5.3. Словарь `DATASET_NORMALIZATION`

Здесь лежат статистики нормализации для каждого датасета:

- `mean`;
- `std`.

Эти значения потом используются в `transforms.Normalize(...)`.

5.4. Функция `load_image_dataset(name, data_root)`

Это главный вход в модуль данных.

Что делает функция по шагам:

1. Проверяет, что имя датасета есть в `DATASET_INFO`.
2. Извлекает `info` для этого датасета.
3. Начинает список преобразований с `transforms.ToTensor()`.
4. Если включена нормализация, добавляет `transforms.Normalize(...)`.
5. Собирает полный `transform = transforms.Compose(tfms)`.
6. Создает обучающий и тестовый датасеты через `info['loader']`.
7. Возвращает `train, test, info`.

Это очень полезный дизайн, потому что `info` дальше передаётся в модель и сценарии. То есть одна функция возвращает не только сами данные, но и метаданные о них.

5.5. Функция `take_subset(dataset, subset_size, seed=SEED)`

Она нужна для ускорения экспериментов. Полный MNIST может быть избыточен для грубого перебора архитектур. Поэтому функция:

1. создаёт генератор `numpy` с фиксированным `seed`;
2. берёт `n = min(subset_size, len(dataset))`;
3. случайно выбирает `n` индексов без повторений;
4. возвращает `Subset(dataset, idx.tolist())`.

Важная идея: сама выборка не копируется, а оборачивается объектом `Subset`, который просто хранит исходный датасет и список индексов.

5.6. Функция `split_train_val(dataset, val_ratio, seed=SEED)`

Она разбивает обучающий набор на `train` и `validation`.

Пошагово:

1. создаётся массив индексов от 0 до `len(dataset) - 1`;
2. индексы перемешиваются;
3. считается `n_val`;
4. первые индексы идут в `train`;
5. хвост идёт в `validation`;
6. возвращаются два объекта `Subset`.

Важно понимать, что `split` здесь делается **не отдельным классом из `sklearn`**, а вручную и очень прозрачно.

5.7. Функция `make_loaders(...)`

Она превращает датасеты в `DataLoader`.

Внутри есть важная строка:

```
use_pin_memory = torch.cuda.is_available() if pin_memory is None else (...)
```

Это означает:

- если `pin_memory` в конфиге стоит в `auto`, то решение принимается автоматически;
- на CPU будет `False`;
- на CUDA будет `True`.

Дальше создаются три loader'а:

- `train_loader` с `shuffle=True`;
- `val_loader` с `shuffle=False`;
- `test_loader` с `shuffle=False`.

Это правильно, потому что порядок примеров важен только для обучения, а не для оценки.

5.8. Функция `plot_random_images(...)`

Эта функция нужна не для обучения, а для визуальной sanity-check проверки.

Она случайно выбирает индексы, отображает изображения в сетке и подписывает их именами классов. Это полезно для двух вещей:

- убедиться, что загрузка вообще работает корректно;
- глазами понять природу данных перед запуском модели.

6. Разбор `cnn.py`

Это центральный файл, потому что в нём находится и архитектура, и логика обучения, и логика оценки.

6.1. Импорты и их смысл

Здесь импортируются:

- `copy` — для сохранения лучшего состояния модели;
- `math` — для расчёта размеров в `is_valid_config`;
- `random`, `numpy`, `torch` — для воспроизводимости и вычислений;
- `time` — для измерения времени обучения;
- `torch.nn as nn` — для описания слоёв и `loss`;
- `SEED` из `config.py`.

6.2. `get_device()`

Функция выбирает устройство в порядке приоритета:

1. `cuda`;
2. `mps`;
3. `cpu`.

Это важно, потому что один и тот же код автоматически запускается и на Mac с Apple Silicon, и на машине с NVIDIA GPU, и на обычном CPU.

6.3. `set_seed(seed=SEED)`

Эта функция централизует фиксацию случайности. Она устанавливает `seed` для:

- встроенного `random`;
- `numpy`;
- `torch`;
- всех CUDA-устройств, если они есть.

Смысл: вызывать одну функцию вместо того, чтобы размазывать этот код по сценарию эксперимента.

6.4. Класс `ConvBlockA`

Это простой свёрточный блок.

Конструктор принимает:

- `in_ch` — число входных каналов;
- `out_ch` — число выходных каналов;
- `conv_k`, `conv_s` — параметры свёртки;
- `pool_k`, `pool_s` — параметры pooling.

Внутри:

```
pad = conv_k // 2
```

Это попытка сохранить пространственный размер при `stride = 1` и нечётном ядре.

Дальше собирается `nn.Sequential` из трёх операций:

```
Conv2d -> ReLU -> MaxPool2d
```

Метод `forward(self, x)` просто применяет этот готовый пайплайн к входу.

6.5. Класс `ConvBlockB`

Это более глубокий блок:

```
Conv2d -> ReLU -> Conv2d -> ReLU -> MaxPool2d
```

Главное отличие от `ConvBlockA` такое:

- внутри одного блока теперь две свёртки;
- вторая свёртка идёт уже из `out_ch` в `out_ch`;
- значит блок способен извлекать более сложные признаки до pooling.

Именно этот тип блока обычно оказывается мощнее, но и тяжелее по вычислениям.

6.6. Класс `ImageCNN`

Это обёртка над всей моделью.

Конструктор принимает:

- `in_channels`;
- `num_classes`;
- `block_type`;
- `channels`;
- `conv_k, conv_s, pool_k, pool_s`;
- `dropout_rate`.

Важный фрагмент:

```
Block = ConvBlockA if block_type == 'a' else ConvBlockB
```

То есть класс не дублирует логику построения двух разных сетей, а выбирает реализацию блока один раз и дальше строит сеть единообразно.

Потом идёт цикл:

```
layers = []
prev = in_channels
for ch in channels:
    layers.append(Block(prev, ch, ...))
    prev = ch
```

Логика цикла очень важна:

- `channels` — это не просто список чисел;
- каждое число означает выходную ширину очередного блока;
- после создания блока `prev` обновляется;
- следующий блок уже получает предыдущий выход как число входных каналов.

После этого строятся две части модели:

- `self.features` — стек свёрточных блоков;
- `self.classifier` — хвост из `AdaptiveAvgPool2d`, `Flatten`, `Dropout`, `Linear`.

Метод `forward(self, x)` выглядит коротко:

```
return self.classifier(self.features(x))
```

Но внутри этой строки скрыт полный путь входного батча через все слои сети.

6.7. `is_valid_config(...)`

Эта функция проверяет, не станет ли пространственный размер карты признаков нулевым или отрицательным после нескольких блоков подряд.

По шагам она:

1. начинает с `s = image_size`;
2. вычисляет размер после свёртки;
3. проверяет, не меньше ли он окна `pooling`;
4. вычисляет размер после `pooling`;
5. проверяет, не упал ли он ниже 1.

Это защита ещё до построения модели. Хорошая идея: ловить плохие конфигурации как можно раньше.

6.8. `make_optimizer(name, params, lr)`

Это маленькая фабрика оптимизаторов. Она решает, какой объект оптимизатора создать по строковому имени.

Плюсы такого подхода:

- сценарий эксперимента не захламляется `if/else` по оптимизаторам;
- список поддерживаемых вариантов централизован;
- второй блок легко переиспользует этот же механизм.

6.9. `confusion_matrix(y_true, y_pred, n_classes)`

Это простая ручная реализация матрицы ошибок на `numpy`.

Алгоритм:

1. создаётся квадратная матрица нулей размера `n_classes x n_classes`;

2. для каждой пары (`true`, `pred`) увеличивается соответствующая ячейка.

Интерпретация:

- строки — истинные классы;
- столбцы — предсказанные классы.

6.10. `train_one_epoch(...)`

Это ядро обучения. Именно здесь модель реально обновляет веса.

Что делает функция:

1. переводит модель в режим обучения `model.train()`;
2. инициализирует аккумуляторы `loss` и `accuracy`;
3. проходит по батчам `for x, y in loader`;
4. переносит батч на устройство;
5. вызывает `optimizer.zero_grad()`;
6. считает прямой проход `out = model(x)`;
7. считает `loss`;
8. делает `loss.backward()`;
9. делает `optimizer.step()`;
10. обновляет агрегированные метрики.

Особенно важно понимать две строки:

```
loss.backward()  
optimizer.step()
```

Первая вычисляет градиенты, вторая обновляет параметры на основе этих градиентов.

6.11. `evaluate(...)`

Эта функция очень похожа на `train_one_epoch`, но есть три важных отличия:

- стоит декоратор `@torch.no_grad()`;
- вызывается `model.eval()`;
- оптимизатор и обратное распространение не используются.

Это уже не обучение, а чистая оценка.

Если `return_preds=True`, функция дополнительно накапливает:

- список истинных меток;
- список предсказанных меток.

Это потом нужно для `confusion matrix`.

6.12. `train_model(...)`

Это самая важная функция высокого уровня в `cnn.py`.

Она управляет всем многоэпохальным обучением и ранней остановкой.

Структура функции:

1. создаётся словарь `history` для кривых обучения;
2. инициализируется лучшее значение `best_val_acc`;
3. сохраняется стартовое состояние `best_state`;
4. запускается цикл по эпохам;
5. на каждой эпохе вызываются `train_one_epoch(...)` и `evaluate(...)`;
6. результаты записываются в `history`;
7. при улучшении `val_acc` сохраняется новое лучшее состояние;
8. при отсутствии улучшения увеличивается счётчик `bad`;
9. при `bad >= patience` происходит early stopping;
10. после цикла загружается `best_state`.

Особенно важна эта строка:

```
best_state = copy.deepcopy(model.state_dict())
```

Нужен именно `deepcopy`, потому что параметры модели меняются во время обучения. Если просто сохранить ссылку, мы не зафиксируем реальное лучшее состояние.

Ещё один важный момент — переменная `epoch_to_target`. Она хранит первую эпоху, на которой `val_acc` достигла целевого `TARGET_ACC`. Это потом используется как дополнительная метрика скорости сходимости.

7. Разбор `block1_main.py`

Этот файл реализует основной исследовательский сценарий лабораторной: подобрать архитектуру CNN из сетки вариантов.

7.1. Импортируемые сущности

Из `config.py` подтягиваются основные параметры эксперимента.

Из `cnn.py` импортируются:

- `ImageCNN`;
- `confusion_matrix`;
- `evaluate`;
- `get_device`;
- `is_valid_config`;
- `make_optimizer`;
- `set_seed`;

- `train_model`;
- `train_one_epoch`.

Из `data.py` импортируются:

- `load_image_dataset`;
- `make_loaders`;
- `plot_random_images`;
- `split_train_val`;
- `take_subset`.

По самим импортам уже видно, что `block1_main.py` ничего фундаментально нового не реализует. Он **орkestрирует** уже готовые строительные блоки.

7.2. Сетка гиперпараметров

В верхней части файла задаются списки вариантов:

```
BLOCK_TYPES = ['a', 'b']
CHANNELS_OPTIONS = [(16, 32), (32, 64)]
CONV_KERNELS = [3, 5]
CONV_STRIDES = [1]
POOL_KERNELS = [2]
POOL_STRIDES = [1, 2]
LEARNING_RATES = [1e-3]
```

Это и есть пространство поиска архитектур.

7.3. `build_grid(image_size=28)`

Функция перебирает все комбинации через `itertools.product(...)`.

Для каждой комбинации:

1. проверяется допустимость через `is_valid_config(...)`;
2. если конфиг валиден, создаётся словарь с параметрами;
3. словарь получает уникальный `id`;
4. конфиг добавляется в список `grid`.

Это означает, что итоговая сетка — это **список словарей**, а не список кортежей. Такой выбор очень удобен: к параметрам потом можно обращаться по именам, а не по индексам.

7.4. `short_name(cfg)` и `cfg_pretty(cfg)`

Обе функции нужны для человекочитаемого вывода.

- `short_name(...)` делает короткую метку для осей графиков и подписей;
- `cfg_pretty(...)` строит подробное строковое описание конфигурации для консоли.

Это не математическая часть, но инженерно это важно: удобный лог помогает анализировать эксперименты.

7.5. `run_one(...)`

Это функция запуска одной конфигурации.

Поток внутри неё такой:

1. фиксируется `seed`;
2. создаётся `ImageCNN` по параметрам `cfg`;
3. модель переносится на устройство через `.to(device)`;
4. создаётся оптимизатор `Adam`;
5. создаётся `CrossEntropyLoss`;
6. считается число параметров `n_params`;
7. вызывается `train_model(...)`;
8. в результирующий словарь добавляются `cfg`, `model`, `n_params`.

Фактически `run_one(...)` превращает словарь гиперпараметров в обученную модель и её метрики.

7.6. `train_final_on_full(...)`

После выбора лучшего конфига модель создаётся заново и обучается уже на полном train-наборе.

Это принципиально важно. Нельзя просто взять модель, обученную на train-части из `split`, и считать её окончательной. После выбора архитектуры лучшее решение — использовать максимально полный `train` для финального обучения.

Внутри этой функции:

1. создаётся новая модель с параметрами `best_cfg`;
2. создаются оптимизатор и `loss`;
3. формируются `full-train` и `full-test loader`'ы;
4. запускается цикл эпох с `train_one_epoch(...)`;
5. после обучения считается test-качество через `evaluate(...)`.

Отличие от `train_model(...)` здесь в том, что нет `validation` и `early stopping`. Это уже не подбор модели, а финальное дообучение выбранной архитектуры.

7.7. `print_results_table(results)`

Это чисто отчётная функция. Она сортирует результаты по `best_val_acc` и печатает удобную таблицу с полями:

- идентификатор конфига;
- тип блока;

- каналы;
- kernel sizes и strides;
- learning rate;
- лучшее val_acc;
- эпоха достижения цели;
- число реально отработанных эпох;
- время обучения.

Очень полезно заметить, что функция не меняет данные. Она только преобразует и печатает их. То есть это чистый presentation-layer.

7.8. Функции построения графиков

В `block1_main.py` несколько функций визуализации:

- `plot_curves(...)`;
- `plot_confusion(...)`;
- `plot_param_compare(...)`;
- `plot_grid_summary(...)`.

Нужно понимать их роли.

7.8.1. `plot_curves(...)`

Строит пары кривых `loss` и `accuracy` по эпохам. Она использует словарь `history`, возвращаемый `train_model(...)`.

7.8.2. `plot_confusion(...)`

Рисует матрицу ошибок через `imshow(...)` и подписывает каждую ячейку значением.

7.8.3. `plot_param_compare(...)`

Группирует результаты по какому-то параметру и рисует среднее и стандартное отклонение `val_acc`.

7.8.4. `plot_grid_summary(...)`

Показывает две сводные картины:

- лучшее `val_acc` по каждой конфигурации;
- эпоху достижения целевого `accuracy`.

7.9. Главная функция `main()`

Это главный маршрут блока 1. Если читать его сверху вниз, можно понять весь жизненный цикл эксперимента.

Пошагово `main()` делает следующее:

1. вызывает `set_seed(SEED)`;
2. определяет устройство через `get_device()`;
3. загружает полный датасет и `info`;
4. берёт train-подвыборку `TRAIN_SUBSET`;
5. делит её на `train_ds` и `val_ds`;
6. берёт test-подвыборку `TEST_SUBSET`;
7. создаёт `train_loader`, `val_loader`, `test_loader`;
8. визуализирует случайные изображения;
9. строит сетку конфигураций;
10. обучает каждую конфигурацию в цикле;
11. печатает таблицу результатов;
12. выбирает лучшую конфигурацию;
13. считает качество лучшей модели на test-подвыборке;
14. запускает финальное обучение на полном train;
15. строит все итоговые графики;
16. печатает статистику достижения `TARGET_ACC`.

Это самая важная функция для понимания всей лабораторной, потому что именно она собирает все остальные модули в единый рабочий pipeline.

8. Разбор `block2_main.py`

Если `block1_main.py` отвечает на вопрос «какая архитектура лучше», то `block2_main.py` отвечает уже на более исследовательские вопросы:

- как разные оптимизаторы влияют на обучение;
- как выглядит переобучение;
- как влияет dropout.

8.1. `BASE_CFG`

Здесь зафиксирована базовая архитектура:

```
BASE_CFG = {
    'block_type': 'b',
    'channels': (32, 64),
    'conv_k': 3, 'conv_s': 1,
    'pool_k': 2, 'pool_s': 2,
}
```

Смысл такой: архитектура больше не ищется, она уже выбрана. Теперь исследуется влияние **остальных факторов**.

8.2. `make_model(info, device, dropout=0.0)`

Это маленькая фабрика модели для второго блока.

Она нужна, чтобы:

- не повторять длинный конструктор `ImageCNN` в нескольких местах;
- централизованно менять только `dropout`;
- всегда строить одну и ту же базовую архитектуру.

8.3. `OPT_VARIANTS`

Это список пар (`optimizer_name`, `lr`).

Дальше второй блок перебирает именно эти пары, чтобы сравнить динамику обучения при одинаковой архитектуре.

8.4. `study_optimizers(...)`

Эта функция по структуре похожа на цикл по конфигурациям из первого блока, но здесь фиксируется архитектура и меняется только оптимизатор с `learning rate`.

Поток внутри такой:

1. создаётся `criterion = nn.CrossEntropyLoss()`;
2. для каждой пары из `OPT_VARIANTS` фиксируется `seed`;
3. создаётся модель `make_model(...)`;
4. создаётся оптимизатор через `make_optimizer(...)`;
5. вызывается `train_model(...)`;
6. в результаты добавляются `opt_name` и `lr`.

Итог — список результатов, у каждого из которых есть история, метрики и сведения об оптимизаторе.

8.5. `plot_optimizer_curves(results, target_acc=TARGET_ACC)`

Эта функция берёт `val_acc` каждого эксперимента и рисует их на одном графике. Именно здесь удобно визуально сравнивать, кто сходится быстрее и стабильнее.

8.6. `run_overfit_demo(info, device)`

Это один из самых учебно ценных фрагментов во всей лабораторной.

Алгоритм:

1. загружается полный датасет;
2. из `train` берётся маленькая подвыборка `OVERFIT_TRAIN`;
3. эта маленькая выборка делится на `train_ds` и `val_ds`;
4. создаются `loader`'ы;
5. запускается обучение без регуляризации (`dropout=0.0`);
6. запускается обучение с `dropout=0.4`;
7. строятся сравнительные графики;
8. печатается финальный `gap` между `train` и `val`.

Важно понять, почему этот эксперимент работает. На маленькой выборке мощная модель легко начинает запоминать данные. Поэтому это почти идеальная среда, чтобы показать переобучение на практике.

8.7. `plot_overfit_compare(res_no, res_dr)`

Эта функция рисует две панели:

- сравнение `loss`;
- сравнение `accuracy`.

Причём на одном графике одновременно показываются:

- `train/val` без регуляризации;
- `train/val` с `dropout`.

Это позволяет увидеть не только итоговые числа, но и то, **как** меняется динамика обучения.

8.8. `main()` второго блока

Поток выполнения здесь такой:

1. фиксируется `seed`;
2. определяется устройство;
3. загружается датасет;
4. берётся `train`-подвыборка и `split train/val`;
5. создаются `loader`'ы;
6. вызывается `study_optimizers(...)`;
7. рисуются кривые оптимизаторов;
8. вызывается `run_overfit_demo(...)`.

То есть второй блок состоит из двух независимых мини-исследований, объединённых общей базовой архитектурой.

9. Какие структуры данных гуляют между функциями

Чтобы действительно хорошо понимать код, полезно отслеживать не только функции, но и **форму данных**, которые передаются между ними.

9.1. 1. `info`

Возвращается из `load_image_dataset(...)` и содержит метаданные о датасете:

- `loader`;
- `classes`;
- `in_channels`;

- `image_size`.

Этот словарь потом используется:

- при создании модели;
- при построении сетки конфигураций;
- при подписывании графиков и `confusion matrix`.

9.2. 2. `cfg`

Это словарь одной конфигурации архитектуры. Он строится в `build_grid(...)`.

Внутри лежат:

- `id`;
- `block_type`;
- `channels`;
- `conv_k`, `conv_s`;
- `pool_k`, `pool_s`;
- `lr`.

По сути `cfg` — это переносимая спецификация одной CNN-конфигурации.

9.3. 3. `history`

Создаётся в `train_model(...)` и содержит списки:

- `train_loss`;
- `val_loss`;
- `train_acc`;
- `val_acc`.

Это главный источник данных для графиков.

9.4. 4. `res` или `result`

Это словарь результата одного обучения. Обычно в нём есть:

- `history`;
- `best_val_acc`;
- `best_epoch`;
- `epoch_to_target`;
- `duration_sec`;
- `trained_epochs`.

В блоке 1 в него дополнительно записываются:

- `cfg`;
- `model`;
- `n_params`.

Во втором блоке добавляются:

- `opt_name`;
- `lr`.

9.5. 5. `final_res`

Это результат финального обучения лучшей модели на полном `train`. Он отличается от обычного результата тем, что содержит уже `test`-оценку и предсказания для `confusion matrix`.

10. Один полный проход выполнения блока 1

Чтобы окончательно склеить всё в голове, полезно проследить один полный сценарий исполнения `block1_main.py`.

1. Python начинает выполнять файл.
2. Срабатывает условие:

```
if __name__ == '__main__':  
    main()
```

3. Запускается `main()`.
4. `main()` вызывает `set_seed` и `get_device`.
5. Потом `load_image_dataset(...)` возвращает `train_full`, `test_full`, `info`.
6. `take_subset(...)` уменьшает `train` до нужного размера.
7. `split_train_val(...)` делит `train` на обучение и валидацию.
8. `make_loaders(...)` создаёт три `loader`'а.
9. `build_grid(...)` создаёт список конфигураций.
10. Для каждого `cfg` вызывается `run_one(...)`.
11. Внутри `run_one(...)` создаётся модель и вызывается `train_model(...)`.
12. Внутри `train_model(...)` на каждой эпохе вызываются `train_one_epoch(...)` и `evaluate(...)`.
13. После окончания обучения `run_one(...)` возвращает результат.
14. `main()` собирает все результаты в список `results`.
15. Из `results` выбирается лучший элемент `best`.
16. Лучшая модель сначала оценивается на `test`-подвыборке.
17. Затем лучший конфиг переобучается на полном `train` через `train_final_on_full(...)`.
18. Строятся итоговые графики и матрица ошибок.

Если понимать этот маршрут, читать код становится намного проще: каждый вызов легко поставить на конкретный этап `pipeline`.

11. Где в коде находятся ключевые решения

Когда изучаешь проект, полезно знать, **в каком именно месте** принимается та или иная логическая развилка.

11.1. Выбор датасета

Находится в `config.py` через `DATASET_NAME`, а применяется в `load_image_dataset(...)`.

11.2. Выбор типа блока

Находится в `cfg['block_type']`, а реализуется внутри `ImageCNN` строкой выбора `ConvBlockA` или `ConvBlockB`.

11.3. Выбор числа каналов

Лежит в `cfg['channels']` и используется в цикле построения `layers` внутри `ImageCNN`.

11.4. Выбор оптимизатора

Находится в вызове `make_optimizer(...)`.

11.5. Решение об early stopping

Происходит внутри `train_model(...)`, в логике сравнения `val_acc` с `best_val_acc`.

11.6. Выбор лучшей архитектуры

Происходит в `block1_main.py` через поиск максимума по `best_val_acc`.

11.7. Демонстрация переобучения

Сосредоточена в `run_overfit_demo(...)`.

12. Типичные вопросы, которые стоит задать себе при чтении кода

Если хочешь действительно глубоко понять лабораторную, читай исходники и параллельно отвечай себе на такие вопросы.

1. Почему `info` возвращается вместе с датасетами, а не вычисляется отдельно?
2. Зачем `cfg` хранится словарём, а не кортежем?
3. Почему лучшая модель определяется по `val_acc`, а не по `test_acc`?
4. Почему в `train_model(...)` хранится `best_state`, а не просто последняя эпоха?
5. Почему `train_final_on_full(...)` заново создаёт модель, а не дообучает уже найденную?

6. Почему для `train_loader` включён `shuffle=True`, а для `val_loader` и `test_loader` нет?
7. Почему `evaluate(...)` отделена от `train_one_epoch(...)`, а не встроена в одну большую функцию?

Если ты можешь ответить на эти вопросы своими словами, значит код уже понят не поверхностно, а по существу.

13. Что в этом коде особенно хорошо сделано

С инженерной точки зрения в лабораторной есть несколько сильных мест.

1. Код `modularized`: разные обязанности разнесены по файлам.
2. Конфигурация вынесена в отдельный модуль и частично в переменные окружения.
3. Сценарии экспериментов переиспользуют общие функции вместо копирования логики.
4. Есть ранняя остановка и корректное разделение `train/val/test`.
5. Есть не только численные метрики, но и графики, и `confusion matrix`.

Это не просто рабочий код, а уже хороший учебный пример аккуратно организованного ML-проекта малого масштаба.

14. Итог

Если кратко, логика лабораторной устроена так:

- `config.py` задаёт правила игры;
- `data.py` готовит данные;
- `cnn.py` определяет, как сеть устроена и как она обучается;
- `block1_main.py` ищет лучшую архитектуру;
- `block2_main.py` исследует оптимизаторы и переобучение.

Главное, что нужно вынести из чтения кода: лабораторная написана как набор переиспользуемых блоков, а не как один длинный скрипт. Именно поэтому её удобно изучать по частям и понимать, как каждая функция вносит вклад в общий результат.

15. Как лучше читать исходники после этого документа

Чтобы материал закрепился, полезно открыть код в таком порядке:

1. `config.py` — просто запомнить все управляющие параметры.
2. `data.py` — проследить путь от имени датасета до `DataLoader`.
3. `cnn.py` — внимательно разобрать `ImageCNN`, `train_one_epoch`, `evaluate`, `train_model`.
4. `block1_main.py` — пройти весь `pipeline` подбора конфигураций.

5. `block2_main.py` — посмотреть, как на той же базе строятся дополнительные исследования.

Если хочешь, следующим шагом уже можно сделать ещё один документ: прям построчный разбор ключевых функций с вставками кода и комментариями по каждой строке.